

# Working With Memory in LangChain

## LangChain Applications with Memory

In LangChain, memory-enabled applications, such as chatbots, maintain conversation context across multiple turns, whereas applications without memory treat each interaction as independent. This creates a smooth, connected experience where each response builds on previous messages.

## Use of Memory in LangChain

In LangChain, memory is used when an application requires access to information from earlier messages, such as in multi-turn conversations or personalized interactions.

## Runnables in LangChain

A `Runnable` is a key building block designed to standardize the execution of different components, such as prompts, models, and chains. This makes it easy to invoke different components using the same pattern, reducing complexity when building workflows.

## LangChain ChatPromptTemplate

The `ChatPromptTemplate` class creates structured prompts for chat-based interactions. It combines system messages, conversation history, and user input into a single, organized format that the language model can process.

```
from langchain_core.prompts import
    ChatPromptTemplate, MessagesPlaceholder

# Create chat prompt template
prompt =
    ChatPromptTemplate.from_messages([
        ("system", "You are a helpful
assistant."),
        MessagesPlaceholder(variable_name="history
"),
        ("human", "{input}")
    ])
```

## LangChain Messages

LangChain uses three primary message types to structure conversations:

- `SystemMessage` : defines the AI assistant's behavior and role
- `HumanMessage` : represents user input
- `AIMessage` : represents the AI's response

This structure creates clear role definitions within conversations.

```
from langchain_core.messages import
    SystemMessage, HumanMessage, AIMessage

messages = [
    SystemMessage(content="You are a
helpful assistant."),
    HumanMessage(content="What's the
weather today?"),
    AIMessage(content="Today's weather is
sunny and warm.")
]

for msg in messages:
    print(f"{msg.type}: {msg.content}")
```

## LangChain MessagesPlaceholder

The `MessagesPlaceholder` component reserves a spot in your prompt template for conversation history. It allows you to dynamically inject previous messages when the prompt is executed, ensuring the language model has access to past context.

```
from langchain_core.prompts import
    ChatPromptTemplate, MessagesPlaceholder

# Create prompt with placeholder for
# conversation history
prompt =
    ChatPromptTemplate.from_messages([
        ("system", "You are a helpful
assistant."),

MessagesPlaceholder(variable_name="history
"),
        ("human", "{input}")
    ])
```

## LangChain InMemoryChatMessageHistory

The `InMemoryChatMessageHistory` class allows you to maintain a log of messages exchanged between the user and the model during a conversation. All messages are stored directly in application memory, making it a convenient choice for tracking temporary conversations during a session.

```
from langchain_core.chat_history import
    InMemoryChatMessageHistory

chat_history =
    InMemoryChatMessageHistory()

chat_history.add_user_message("Hello, how
can I help you today?")
chat_history.add_ai_message("I'm doing
well, thank you!")

for message in chat_history.messages:
    print(f"{message.type}:
{message.content}")
```

## LangChain `session_id`

A `session_id` acts as a unique identifier for each conversation, ensuring messages are associated with the correct conversation history. Each session maintains its own separate history, preventing conversations from mixing.

```
# Use session_id to identify different
conversations

response = conversation_chain.invoke(
    {"input": "Hello!"},
    config={"configurable": {"session_id":
"session-123"}}
)
```

## LangChain Memory Integration

In LangChain, memory is added by passing a memory object during chain creation and including a placeholder in the prompt template for conversation history. This integration allows the model to access past interactions when generating responses.

## LangChain `RunnableWithMessageHistory`

The `RunnableWithMessageHistory` class in LangChain enhances your existing `Runnable` by automatically managing the message history. It loads conversation history before processing and saves new messages after processing, maintaining context across conversation turns without manual tracking.

```
from langchain_core.runnables.history
import RunnableWithMessageHistory

# Assuming base_chain and
get_session_history are defined
conversation_chain =
RunnableWithMessageHistory(
    base_chain,
    get_session_history,
    input_messages_key="input",
    history_messages_key="history"
)
```

## LangChain Memory Setup

For memory to work correctly in LangChain, the memory component must match the chain's configuration. This means using consistent variable names and appropriate memory types. Proper alignment prevents runtime errors and ensures smooth data flow through your chain.

## Window Size in LangChain

The window size parameter controls the number of recent message pairs (exchanges) to keep in memory. Setting `k=3` keeps only the last 3 exchanges ( 6 messages total), discarding older ones. This prevents memory from growing indefinitely.

```
from langchain_core.chat_history import
InMemoryChatMessageHistory

WINDOW_SIZE = 3 # Keep last 3 exchanges
window_memory =
InMemoryChatMessageHistory()

# Trim to window size
window_memory.messages =
window_memory.messages[-(WINDOW_SIZE *
2):]
```

## Hybrid Memory in LangChain

Hybrid memory achieves a balance between retaining long-term context and managing memory constraints. It stores summaries of older conversations while keeping recent messages in full detail. This approach ensures that the system maintains essential context without overwhelming memory resources.

```
from langchain_core.prompts import
    ChatPromptTemplate, MessagesPlaceholder
from langchain_core.chat_history import
    InMemoryChatMessageHistory

# Long-term summarized memory
summary = "Project: TaskMaster app. Team:
Alex, Jordan, Sam. Sprint 1 done."

# Short-term window memory
recent_memory =
    InMemoryChatMessageHistory()
recent_memory.add_user_message("What's
left for Sprint 2?")
recent_memory.add_ai_message("We need to
finalize the API and write tests.")

# Prompt that uses BOTH summary and recent
messages
prompt =
    ChatPromptTemplate.from_messages([
        ("system", "Project Summary:
{summary}"),

        MessagesPlaceholder(variable_name="recent_
messages"),

        ("human", "{input}")
    ])

# Inject hybrid memory into the prompt
messages = prompt.format_messages(
    summary=summary,

    recent_messages=recent_memory.messages,
    input="What should I work on next?"
)
```

```
for msg in messages:
    print(f"{msg.type}: {msg.content}")
```

## Entity Memory in LangChain

Entity-based memory extracts and stores specific information (like names, locations, or facts) separately from the conversation history. This approach keeps important details accessible without storing complete chat histories, ensuring long-term retention of critical information and improving efficiency.

```
entity_store = {}

entity_store["John Doe"] = {
    "occupation": "Engineer",
    "location": "New York"
}

entity_details = entity_store.get("John
Doe")
print(entity_details)
```

## LangChain Summary Memory

`ConversationSummaryMemory` is a memory type in LangChain that uses a language model to create and maintain a running summary of a conversation. It reduces token usage by condensing past exchanges while preserving essential context and key details.

```
from langchain.memory import
ConversationSummaryMemory
from langchain.llms import OpenAI

# Initialize memory with an LLM summarizer
llm = OpenAI(model="gpt-4")
summary_memory =
ConversationSummaryMemory(llm=llm)

# Add conversation entries
summary_memory.add_memory("User: Can you
explain LangChain?")
summary_memory.add_memory("AI: LangChain
is a framework for building LLM-based
applications.")
summary_memory.add_memory("User: Summarize
what we discussed.")

# Retrieve the summarized conversation
conversation_summary =
summary_memory.get_memory()
print(conversation_summary)

# Output shows a concise summary of key
information.
```



## LangChain ConversationBufferMemory

`ConversationBufferMemory` is a memory type in LangChain that stores the full conversation history exactly as written. It allows models to access and reference every previous message during an interaction.

```
from langchain.memory import
ConversationBufferMemory

# Initialize the memory buffer
dialogue_memory =
ConversationBufferMemory()

# Add conversation entries
dialogue_memory.add_memory("User: What is
LangChain?")
dialogue_memory.add_memory("AI: LangChain
is a framework designed to ...")
dialogue_memory.add_memory("User: How does
it handle memory?")

# Retrieve full conversation
dialogue_history =
dialogue_memory.get_memory()
print(dialogue_history)

# Output shows the complete dialogue
stored in memory.
```

## LangChain Conversation

LangChain provides convenience methods to add messages to the conversation history. Use `.add_user_message()` for user inputs and `.add_ai_message()` for AI responses. These methods automatically create the correct message types and preserve context.

```
from langchain_core.chat_history import
InMemoryChatMessageHistory

# Create conversation history
conversation =
InMemoryChatMessageHistory()

# Add messages using convenience methods
conversation.add_user_message("Hello, how
are you?")
conversation.add_ai_message("I'm good,
thank you! How can I help you today?")

# Inspect conversation history
for msg in conversation.messages:
    print(f"{msg.type}: {msg.content}")
```

 [Print](#)